

JARVIS

VOICE OS ASSISTANT

A Modular, Offline-First AI Voice Assistant for Windows 11

Project Technical Report

Version 0.7.0

Author: Manthan Dhanraj

B.Tech · Sharda University

Table of Contents

Table of Contents 2
1. Abstract 4
Keywords 4
2. Introduction 4
2.1 Background 4
2.2 Motivation 4
2.3 Problem statement 5
2.4 Objectives 5
2.5 Report organisation 5
3. Requirements 5
3.1 Functional requirements 5
3.2 Non-functional requirements 5
3.3 Target environment 6
4. System Overview 6
4.1 Processing pipeline 6
4.2 Design principles 6
5. Architecture 7
5.1 The module lifecycle 7
5.2 Abstraction boundaries 7
5.3 Data flow and control flow 8
6. The Voice Subsystem 8
6.1 Microphone capture 8
6.2 Wake-word detection 8
6.3 Speech-to-text 8
6.4 Text-to-speech 9
7. The Command and Intelligence Layer 9
7.1 The router 9
7.2 The command registry 9
7.3 Intent parsing and target extraction 9
7.4 The AI brain 9
8. Automation Services 10
8.1 The file-management safety boundary 10
8.2 Fallbacks in practice 10
9. Memory and Safety 10
9.1 Persistent memory 10
9.2 The risk model 11
9.3 The power-action incident 11
10. The Graphical Dashboard 11
11. Interaction Modes 11
12. Key Design Decisions 12
13. Deployment and Real-World Debugging 12
14. Testing and Validation 13
14.1 Routing smoke tests 13
14.2 Live voice trials 13
14.3 Diagnostic instrumentation 13
15. Results, Challenges, and Future Work 14

15.1 Results..... 14
15.2 Challenges..... 14
15.3 Future work..... 14
15.4 Conclusion..... 14
16. Comparison with Cloud Assistants..... 14
16.1 Positioning..... 15
17. Software Design Patterns Employed..... 15
17.1 Why patterns matter here..... 15
Appendix A — Command Reference..... 17
Appendix B — Subsystem Map..... 17
Appendix C — Configuration Reference..... 18
Appendix D — Dependency Overview..... 18
Appendix E — Glossary..... 18
Appendix F — Interaction Walkthrough..... 19
Appendix G — Lessons Learned..... 19

1. Abstract

JARVIS is a modular, offline-first voice assistant for Windows 11 that lets a user operate their computer hands-free through natural spoken commands. The system listens continuously for a wake word, transcribes the user's speech locally, interprets the request through either a deterministic command layer or a local large language model, performs the corresponding action on the operating system, and responds in a natural neural voice.

Unlike cloud assistants, every core capability — speech recognition, speech synthesis, and language understanding — executes entirely on the user's machine. This makes the assistant private by design, usable without an internet connection for its core loop, and fully controllable at the source-code level. The project is organised into fourteen cooperating subsystems built behind clean abstract interfaces, so any individual engine can be replaced without disturbing the rest of the system.

This report documents the motivation, requirements, architecture, subsystem design, implementation, testing, and real-world debugging of the project at version 0.7.0. It is intended both as a record of the engineering and as a guide for anyone extending the system.

Keywords

Voice assistant, offline speech recognition, wake-word detection, neural text-to-speech, local large language model, Windows automation, modular architecture, risk-gated actions, Python.

2. Introduction

2.1 Background

Conversational voice assistants are now embedded in phones, speakers, and cars. The most capable of them, however, are cloud services. They record audio and send it to remote data centres for processing, they cannot function without connectivity, and their internals are closed to the people who depend on them. For a user who wants privacy, offline capability, or the freedom to extend the assistant, these systems are fundamentally limiting.

Advances in efficient on-device models have changed what is possible. Compact speech-recognition models run in real time on a laptop CPU; neural text-to-speech voices small enough to ship as a few tens of megabytes now sound convincingly human; and quantised large language models run locally through lightweight runtimes. Together these make a fully local assistant not just possible but practical.

2.2 Motivation

JARVIS was built to exploit exactly this shift: an assistant that controls a personal computer, understands natural language, and speaks back, while keeping all computation and data on the device. The name is an homage to the fictional assistant from popular culture, and while no real system matches that fiction, the felt experience — natural understanding, a courteous tone, a capable voice — is an achievable target.

The project was also a deliberate engineering exercise. Rather than a single script that grows unmanageable, the aim was to design a large system from clean architectural principles: cooperating subsystems, clear interfaces, and graceful failure. Building, deploying, and then debugging it on real hardware was itself a core objective.

2.3 Problem statement

Build a voice-controlled assistant for Windows 11 that (a) activates hands-free, (b) recognises speech without the cloud, (c) understands both fixed commands and open-ended language, (d) controls the operating system across many domains, (e) replies in a natural voice, (f) refuses to perform dangerous actions without confirmation, and (g) remembers context — all while remaining modular enough to evolve.

2.4 Objectives

- Activate hands-free through a spoken wake word with minimal false triggers.
- Recognise speech offline, with automatic fallback across hardware.
- Interpret both fixed commands and free-form natural language.
- Control applications, volume, brightness, browser, files, development tools, and games.
- Respond in a natural, human-sounding voice with graceful degradation.
- Gate risky actions behind an unbypassable confirmation and safety model.
- Persist facts and context across sessions.
- Offer text, voice, and graphical interaction modes over one shared core.

2.5 Report organisation

Section 3 states requirements. Section 4 gives a system overview and design principles. Section 5 presents the architecture. Sections 6 to 10 describe each major subsystem in turn. Section 11 covers the interaction modes. Section 12 discusses key design decisions. Section 13 documents deployment and real-world debugging. Section 14 describes testing. Section 15 presents results, challenges, and future work. Appendices list the command reference and the module map.

3. Requirements

3.1 Functional requirements

ID	Requirement
F1	Detect the wake word continuously and acknowledge activation.
F2	Transcribe a spoken command to text offline.
F3	Route a command to either a deterministic handler or the language model.
F4	Execute OS actions: apps, system controls, web, files, dev, games.
F5	Speak a natural-language reply.
F6	Ask for confirmation before destructive or high-risk actions.
F7	Store and recall user facts and conversational context.
F8	Provide text, voice, and GUI modes.

3.2 Non-functional requirements

ID	Requirement
N1	Privacy — core loop performs no network calls.

ID	Requirement
N2	Resilience — every engine degrades gracefully rather than crashing.
N3	Responsiveness — usable latency on a CPU-only machine.
N4	Modularity — engines replaceable behind abstract interfaces.
N5	Observability — structured logging at every pipeline stage.
N6	Safety — the risk boundary derives from command category, not model output.

3.3 Target environment

The system targets Windows 11 on Python 3.14, with a microphone and speakers, roughly two gigabytes of free disk for the speech and language models, and an optional NVIDIA GPU for accelerated transcription. The reference development machine used an RTX-class GPU, an eight-gigabyte VRAM budget, and sixteen gigabytes of system memory.

4. System Overview

At the highest level JARVIS is a pipeline flanked by two supporting services. Audio enters through the microphone; the wake-word detector decides when the user is addressing the assistant; the speech-to-text engine converts the utterance to text; the router decides whether a deterministic command or the language model should handle it; the resulting action is executed against the operating system; and the reply is spoken back through the text-to-speech engine. A memory service supplies context, and a risk gate enforces safety.

4.1 Processing pipeline

```
Microphone
  -> Wake Word Detector   (rolling-window keyword spotting)
  -> Speech-to-Text        (faster-whisper, GPU->CPU fallback)
  -> Router                (keyword match first, LLM fallback)
       -> Command Layer     (deterministic OS actions)  -> Risk Gate
       -> AI Brain          (local LLM via Ollama)
  -> Action Execution      (automation services)  <-> Memory (SQLite)
  -> Text-to-Speech        (Piper neural -> SAPI5 -> pyttsx3)
  -> Speaker
```

4.2 Design principles

Offline-first

The core loop must not depend on the network. Speech recognition, synthesis, and language understanding all run locally, so the assistant works on a disconnected machine and never transmits user audio.

Modularity through interfaces

Every engine that could have more than one implementation hides behind an abstract base class. The speech-to-text engine, the text-to-speech engine, the wake-word detector, the language-model engine, and the input/output channel are all defined this way, so a concrete implementation can be swapped at construction without touching callers.

Graceful degradation

Each engine has a fallback. Transcription falls back from GPU to CPU; speech falls back from neural to system voices; the language model, when absent, leaves the fixed commands fully functional. The system loses quality under adverse conditions rather than failing outright.

Safety by default

Destructive actions are gated behind confirmation, and the most dangerous class — power actions — is simulated unless explicitly armed. The safety decision is structural, not advisory.

Observability

A shared logging utility emits structured, timestamped records from every subsystem, which proved essential during hardware debugging: the fix for each field problem began with a log line.

5. Architecture

The codebase is organised into fourteen subsystems, each in its own package, all built on a shared module lifecycle. The table below maps packages to responsibilities.

Package	Responsibility
core/	Assistant lifecycle, session loop, router, risk model, I/O channel base
voice/	Microphone capture, wake word, speech-to-text, text-to-speech, voice channel
brain/	Command registry, intent parsing, conversation, and the LLM engine
automation/	Windows, browser, developer, gaming, and file-management services
memory/	SQLite store, data models, and context resolution
security/	Confirmation prompts and risk enforcement
gui/	Tkinter dashboard and optional system tray
config/	Central typed settings for every subsystem
utils/	Logging, custom exceptions, and text-normalisation helpers

5.1 The module lifecycle

A common base module defines an initialise / mark-ready / shutdown lifecycle that every subsystem follows. The assistant brings components up in dependency order at start-up and tears them down in reverse at exit, so resources such as the microphone stream, the model handles, and the database connection are always released cleanly. This uniformity is what allows the same core to power the text, voice, and GUI modes unchanged.

5.2 Abstraction boundaries

Five engine families are defined as abstract base classes: speech-to-text, text-to-speech, wake word, the language-model client, and the input/output channel. Each declares the minimal contract its callers rely on. Concrete implementations — Whisper for transcription, Piper for synthesis, the keyword detector for wake word, the Ollama client for the LLM, and the console and voice channels for I/O — implement these contracts. Because callers depend only on the interface, an implementation can be replaced or a new one added without ripple effects.

5.3 Data flow and control flow

Control flows top-down through the pipeline of Figure 1, but two cross-cutting flows run alongside it. The memory service is consulted when the language model needs context and updated when the user states a fact to remember. The risk gate intercepts every command between the router and execution, and may pause the pipeline to request confirmation before allowing an action to proceed.

6. The Voice Subsystem

6.1 Microphone capture

Audio is captured through the sounddevice library at sixteen kilohertz in mono, the format the speech models expect. The microphone module exposes two capture modes: a fixed-length recording used internally, and an energy-based utterance capture used for commands. In the latter, recording begins when the input energy crosses a threshold calibrated against ambient noise at start-up, and ends after a configured period of silence, so the user does not need to signal when they have finished speaking.

6.2 Wake-word detection

The wake-word detector continuously spots the phrase that addresses the assistant. Reaching a reliable design took three iterations, each documented in Section 13, and the final approach is worth describing in full because it is the heart of the hands-free experience.

A single input stream is opened and driven by a callback that the audio library invokes from its own audio thread. That callback continuously appends audio into a fixed-length rolling buffer. Meanwhile the main loop periodically takes a snapshot of the buffer and transcribes it. Because capture never stops while transcription runs, no speech is lost to a dead gap. Consecutive snapshots overlap, so a wake word spoken across a snapshot boundary is still caught. Before transcribing, the loop checks the audio energy and skips silent windows, which saves processing and prevents the model from inventing words out of background noise.

Matching is intentionally lenient. The transcript is first checked for the exact phrase, then for characteristic substrings, and finally with a fuzzy word comparison against the assistant's name and its common mishearings, because small speech models frequently render the name imperfectly. This tolerance is what reduced activation from many repeated attempts to a single reliable utterance.

6.3 Speech-to-text

Transcription uses faster-whisper, an efficient implementation of the Whisper speech-recognition model. The engine resolves the best available device at start-up, preferring the GPU and falling back to the CPU. A subtle failure mode motivated a specific design choice: on a machine advertising a GPU but missing the CUDA runtime libraries, the model loads successfully but fails only on the first real transcription, which would otherwise crash the assistant mid-conversation. To force this failure to surface early, the engine performs a silent warm-up transcription of a short buffer of zeros immediately after loading, and if that fails on the GPU it rebuilds transparently on the CPU. A second guard in the transcription path repeats this fallback if a GPU error ever slips through at runtime.

The language and model size are configurable. The reference configuration forces English and uses a compact model, trading some accuracy for the low latency that keeps the wake loop responsive on a CPU.

6.4 Text-to-speech

Speech synthesis is layered across three backends in priority order, so the assistant always has a working voice. The preferred backend is Piper, a compact neural voice built on the VITS architecture and run through the ONNX runtime; it produces the natural, human-sounding output that defines the assistant's character. If a Piper voice model is not present, the system falls back to the Windows SAPI5 voice driven directly through COM, which is robust and synchronous. As a final resort it uses the pyttsx3 library, creating a fresh engine for each utterance to avoid a driver-loop stall that otherwise silences repeated calls. The layering means a missing dependency lowers voice quality but never removes the voice.

7. The Command and Intelligence Layer

7.1 The router

When an utterance has been transcribed and cleared of any leaked wake-word fragments, the router first attempts to match it against the deterministic command registry. Fixed commands are fast, predictable, and safe to classify by category. Only when no command matches does the router fall back to the language model. This ordering is deliberate: it keeps everyday actions instant and reserves the comparatively expensive language model for genuinely open-ended requests. It also means the safety classification never depends on the model, because anything the model handles is conversational rather than an OS action.

7.2 The command registry

Commands are self-describing objects grouped by domain. Each declares the keywords or patterns it matches, a human-readable description used by the help system, and a risk level, and implements the logic to execute itself and to describe what it is about to do. The registry assembles these into an ordered list, and the router consults them in order, so more specific commands can take precedence over more general ones. Adding a capability means writing and registering a new command object; the router itself never changes.

A representative example is web handling, where the ordering rule ensures that an explicit request to open a site opens the site, while a request to play media routes to the media handler. This was tightened after early testing showed an open request being misrouted into a media search.

7.3 Intent parsing and target extraction

Many commands need not just to match but to extract a target — the application to open, the query to search, the song to play. A shared set of text helpers normalises the transcript and strips trigger and filler words, leaving the salient target. Because transcripts from a small model are noisy, this normalisation is forgiving of extra words and punctuation.

7.4 The AI brain

The language model runs locally through Ollama using a model from the qwen2.5 family. The brain wraps the model client behind the same abstraction as every other engine, so it can be swapped or disabled. At start-up the brain probes the local Ollama endpoint; if the model is reachable it comes online, and if not, the brain logs that it is offline and the assistant continues to serve fixed commands. When online, the model interprets requests the fixed layer cannot, giving the user the freedom to speak naturally. A

command-line flag disables the brain entirely for environments where only deterministic behaviour is wanted.

8. Automation Services

The automation layer is where the assistant affects the operating system. Each domain is a service with a clear surface and its own fallback strategy, and all are aggregated behind a single automation service that the commands call into.

Service	What it does	Fallback strategy
Apps	Launch and close applications by name	Start-menu index of installed programs
Volume	Set, mute, and adjust system volume	Media-key emulation when the audio library is absent
Brightness	Set display brightness	WMI when the primary library is missing
Power	Shutdown, restart, sign-out	SAFE MODE — simulated unless explicitly armed
Web	Open sites, search, play media	Default browser and configured search engine
Dev	Open a project in the editor, run it, create a venv	Configured editor and terminal executable paths
Games	Launch Steam, Epic, and Riot titles	Store front-ends when a title is not cached
Files	Find, open, move, copy, and organise files	Whitelisted roots; deletions go to the Recycle Bin

8.1 The file-management safety boundary

File operations are the most dangerous automation domain, so they are the most constrained. Operations are confined to a set of whitelisted user folders, and deletions are routed to the Recycle Bin rather than being permanent, so a misheard command cannot cause irreversible data loss.

8.2 Fallbacks in practice

Every service was written assuming its ideal dependency might be missing on a given machine, an assumption vindicated during deployment. The volume service falls back to emulating media keys when the preferred audio library is unavailable, and the brightness service falls back to a system-management interface. These fallbacks mean the assistant is useful immediately on a fresh machine, before every optional dependency is installed.

9. Memory and Safety

9.1 Persistent memory

A memory service backed by an on-disk SQLite database stores facts the user asks the assistant to remember and enough conversational context to resolve later references. Because the store is persistent,

a user can establish context in one session and rely on it in the next — the assistant does not forget when it is closed. The service is wrapped so that it can be disabled with a flag, and so that a corrupt or missing database degrades to an empty memory rather than a crash.

9.2 The risk model

Every action carries a risk level of SAFE, MEDIUM, or HIGH. SAFE actions, such as opening an application or changing the volume, execute immediately. MEDIUM and HIGH actions — anything destructive or system-level — are routed through a confirmation step before they run. The decisive design choice is that the risk level is a property of the command's category, fixed in code, rather than something inferred at runtime by the language model. Because of this, the safety boundary cannot be argued around: no phrasing of a request can lower its risk class.

9.3 The power-action incident

During early testing, a spoken command intended only as a test caused the development laptop to actually shut down. The response reshaped the project's safety philosophy. Power actions — shutdown, restart, and sign-out — were redesigned to ship in a SAFE MODE in which they are simulated and logged rather than executed, unless the user explicitly arms them through an environment variable. When armed, the delay before a real power action was lengthened to give the user time to cancel. Separately, phrases such as turning off music were prevented from ever being misrouted into a system power action. The incident is a reminder that in a system that can act on the world, the default must be inaction.

10. The Graphical Dashboard

Beyond the voice and text modes, the assistant offers a dashboard built with Tkinter. It presents three areas: a live status panel showing which subsystems are ready, a scrolling chat log of the conversation, and a panel of recent actions. An optional system-tray integration lets the assistant run quietly in the background and be summoned when needed. Because the dashboard is only a view over the same assistant core used by the other modes, its behaviour is identical to them; it adds a window, not a second implementation.

11. Interaction Modes

The same core supports three ways of interacting, selected by a command-line option.

Mode	Command	Description
Text	python main.py	Type commands and read replies; useful for testing and for noisy environments.
Voice	python main.py --mode voice	Full hands-free operation via the wake word and spoken replies.
GUI	python main.py --mode gui	The dashboard window with live status, chat, and recent actions.

Two further flags modify behaviour: one disables the language model so that only deterministic commands run, and another disables persistent memory. These are valuable both for constrained environments and for isolating behaviour during testing.

12. Key Design Decisions

Keyword-first, LLM-fallback routing

Routing fixed commands before the language model keeps common actions instant and deterministic and confines the model to open-ended language. It also keeps the safety classification independent of the model.

Category-based risk, not model-based

Deriving risk from the command category rather than model output makes the safety boundary structural and unbypassable, at the cost of some flexibility — a deliberate trade in favour of safety.

Layered engines with fallbacks

Each perception engine has an ordered set of backends. This adds a little complexity but buys the resilience that let the assistant run on a machine where several ideal dependencies were unavailable or blocked.

Callback-driven wake capture

Letting the audio library drive capture from its own thread avoided both a threading pitfall on Windows and the dead-gap problem of fixed-clip recording, and was the single change that made activation reliable.

English-first configuration

Although the assistant was conceived as multilingual, the compact speech model handled a single forced language far more consistently on short clips than it handled automatic language detection, so the reference configuration standardises on English.

13. Deployment and Real-World Debugging

Moving from a clean codebase to a working assistant on real Windows hardware surfaced a sequence of instructive problems. Each is described below with its cause and resolution, because together they account for much of the system's present robustness.

13.1 Missing native audio backend

On the first run the audio stack failed to import because a compiled backend it depends on was absent. Reinstalling that backend against the correct build restored the import. Lesson: native dependencies must be verified independently of their Python wrappers.

13.2 GPU speech-to-text crash

The assistant booted fully but crashed on the first transcription with a missing-CUDA-library error, because the model had loaded on the GPU while the runtime libraries were absent. The engine was rewritten to warm up immediately at start-up and to fall back to CPU execution automatically, converting a mid-conversation crash into a silent, logged fallback.

13.3 Silent replies

Replies appeared as text but produced no audio. Standalone tests showed the speech library worked, so the fault lay in reusing a single engine across the long-running session loop, which stalled its internal event loop. The fix was to synthesise through the system voice directly and, in the fallback path, to create a fresh engine per call.

13.4 Blocked media library

Windows Application Control blocked a media-decoding library bundled with the speech package, which broke the whole package's import. Since the assistant only ever feeds raw audio arrays and never decodes media files, a lightweight stub was registered in place of the blocked component, keeping the package importable while leaving the unused path clearly disabled.

13.5 Unreliable wake word

The original wake detector recorded fixed clips and was deaf while transcribing, so roughly half of all speech fell into a dead gap and the user had to repeat the wake word many times. A first rewrite added a reader thread but hit a Windows-specific problem in which audio read from a background thread returned pure silence. The final callback-driven rolling-window design, described in Section 6.2, eliminated both problems.

13.6 Robotic voice

The default system voice sounded mechanical, which undercut the assistant's character. Integrating the Piper neural voice, with the system voices retained as fallbacks, gave replies a natural, human quality.

13.7 The role of version control

After an earlier loss of the entire project folder, the codebase was placed under version control and pushed to a remote repository, with large model files and the virtual environment deliberately excluded. This protects the work and makes each subsequent change a small, recorded increment.

14. Testing and Validation

The system was validated through a combination of routing smoke tests and live hardware trials.

14.1 Routing smoke tests

A suite of representative utterances was run through the router in text mode to confirm that each mapped to the intended command and risk class, and that ambiguous phrasings resolved to the correct handler. These tests are fast, require no audio, and catch regressions in the command layer immediately; they were the first line of defence when the web-command ordering was tightened.

14.2 Live voice trials

Voice-mode trials on the target hardware validated the parts that cannot be exercised without real audio: microphone calibration, wake-word reliability, transcription accuracy, and the responsiveness of the spoken reply. Instrumentation added during these trials — logging the measured input energy and the exact transcript for every window — turned wake-word tuning from guesswork into a data-driven process.

14.3 Diagnostic instrumentation

A standalone microphone probe was used to confirm that the input device was live and that the captured signal was strong, which isolated a period of apparent deafness to the capture path rather than the hardware. This separation of concerns — is the microphone producing signal, is the signal reaching the code, is the code transcribing it — structured the debugging of the voice pipeline.

15. Results, Challenges, and Future Work

15.1 Results

At version 0.7.0 the assistant reliably wakes on its wake word, transcribes commands offline, understands both fixed and free-form requests, performs actions across eight automation domains, remembers context between sessions, and replies in a natural neural voice. It offers text, voice, and graphical modes over one shared core, and enforces a safety model that gates every risky action. The perception-to-action loop is complete.

15.2 Challenges

- Balancing wake-word sensitivity against false activations on a compact model.
- Making every engine degrade gracefully across widely varying hardware.
- Keeping end-to-end latency low enough on a CPU-only machine to feel immediate.
- Designing a safety boundary that no phrasing of a request can bypass.
- Diagnosing platform-specific failures such as blocked libraries and thread-bound audio.

15.3 Future work

- Speaker-based voice authentication, the deferred sixth stage of the plan.
- A GPU-accelerated transcription preset using a larger model for higher accuracy.
- Richer, longer-horizon conversational memory and summarisation.
- A packaged installer for one-click setup on a fresh machine.
- Expanded automation domains and user-defined command plug-ins.

15.4 Conclusion

JARVIS demonstrates that a private, extensible, and genuinely useful voice assistant can run entirely on a personal computer. More than any single feature, its value lies in an architecture of cooperating, individually replaceable subsystems, and in the resilience won by debugging it on real hardware. That combination — clean structure and hard-earned robustness — is a foundation the system can grow on, toward the natural and capable assistant it is named after.

16. Comparison with Cloud Assistants

It is useful to situate JARVIS against the mainstream cloud assistants it deliberately departs from. The comparison is not about matching their breadth, which is backed by vast infrastructure, but about the trade-offs each design makes.

Dimension	Cloud assistants	JARVIS
Processing location	Remote data centres	Entirely on the local machine
Connectivity	Required for core function	Not required for the core loop
Privacy	Audio leaves the device	Audio never leaves the device
Extensibility	Closed; limited developer surface	Open source; new commands are plain objects

Dimension	Cloud assistants	JARVIS
Language model	Large, cloud-hosted	Compact, local, swappable
Breadth of skills	Very large	Focused on OS control and conversation
Failure mode	Unavailable when offline	Degrades gracefully, stays useful

The pattern is consistent: JARVIS trades raw breadth for privacy, control, and resilience. For its intended use — a personal, inspectable assistant that operates the user's own computer — those are the right trades.

16.1 Positioning

JARVIS is closest in spirit to open, self-hosted assistant projects rather than to commercial products. What distinguishes it within that space is the tight integration with the Windows operating system, the explicit risk model, and the layered engines that let it run on modest or partially configured hardware. It is designed to be a foundation a single developer can understand end to end and extend confidently.

17. Software Design Patterns Employed

The architecture leans on a handful of established patterns, which is part of why it remains comprehensible at fourteen subsystems.

Strategy

Each engine family — speech-to-text, text-to-speech, wake word, language model, and I/O channel — is a strategy interface with interchangeable concrete implementations selected at construction. This is what makes the engines swappable.

Registry

Commands register themselves into an ordered registry that the router consults. New behaviour is added by registration rather than by modifying a central dispatcher, keeping the router closed to modification but open to extension.

Facade

The automation layer presents a single aggregated service over many domain services, so the command layer calls one coherent surface instead of reaching into each subsystem individually.

Template method

The shared module lifecycle — initialise, mark ready, shut down — is a template the concrete subsystems fill in, which is what lets the assistant manage every component uniformly.

Chain of responsibility

Routing itself is a chain: the utterance is offered to each command in turn and, failing all of them, to the language model, so responsibility passes along the chain until something handles it.

17.1 Why patterns matter here

None of these patterns was adopted for its own sake. Each solves a concrete problem the project faced — swapping engines, adding commands, presenting a clean surface, managing lifecycles uniformly, and

routing cleanly — and together they are the reason the system could grow to its present size and still be debugged one subsystem at a time.

Appendix A — Command Reference

A representative selection of spoken commands, grouped by domain.

Domain	Example utterances
Apps	"open notepad" · "close chrome" · "open calculator"
System	"volume 30" · "mute" · "brightness 50"
Web	"open youtube" · "play arijit singh" · "search python decorators"
Coding	"open my project in vs code" · "run the project" · "create a venv"
Gaming	"open steam" · "launch <game name>"
Files	"find my resume" · "open downloads" · "organize this folder"
Memory	"remember my roll number is …" · "what is my roll number"
Conversation	any open-ended question, answered by the local language model

Appendix B — Subsystem Map

The fourteen subsystems and their packages.

Subsystem	Package(s)
Core & session	core/ (assistant, session, router, risk, io)
Configuration	config/ (settings)
Utilities	utils/ (logger, exceptions, text)
Microphone	voice/audio/
Wake word	voice/wakeword/
Speech-to-text	voice/stt/
Text-to-speech	voice/tts/
Voice channel	voice/voice_io.py
Commands	brain/commands/ (system, web, coding, gaming, files, memory, builtin)
Intelligence	brain/ (ai_brain, conversation, intent, llm/)
Automation	automation/ (windows, browser, dev, games, files)
Memory	memory/ (store, models, context, service)
Security	security/ (confirmation)
GUI	gui/ (dashboard)

— End of Report —

Appendix C — Configuration Reference

All behaviour a user is likely to adjust lives in one typed settings module. The table lists the principal groups and what they control.

Setting group	Controls
Microphone	Sample rate, block size, silence threshold, calibration multiplier, start timeout, silence hangover, maximum utterance length, and pre-roll padding.
Speech-to-text	Model size, device selection, compute type, forced language, and beam size.
Wake word	The wake phrases, the rolling-window length, and the polling cooldown.
Text-to-speech	Speech rate, volume, and the preferred voice hint used to select among installed voices.
AI brain	The Ollama model name, the endpoint URL, and the fallback model.
Memory	The SQLite database path and whether persistence is enabled.
Automation	Per-domain settings such as the default browser, the default search engine, and the editor and terminal executables.

Centralising these values means that adapting the assistant to a new machine or preference is a matter of editing one file, not hunting through the code. It also makes the reference configuration self-documenting: a reader can see, in one place, every knob the system exposes.

Appendix D — Dependency Overview

The assistant is built on a small set of well-chosen libraries, each isolated behind the assistant's own abstractions so that it can be replaced.

Dependency	Role in the system
faster-whisper	Efficient offline speech recognition.
Piper	Neural text-to-speech voice.
Ollama client	Access to the local large language model.
sounddevice	Microphone capture and audio playback.
comtypes / SAPI5	The Windows system voice used as a speech fallback.
SQLite	Persistent storage for memory.
Tkinter	The graphical dashboard.

Because every dependency sits behind an interface, the list is a set of current choices rather than a set of hard constraints. The speech-recognition engine could be replaced with another Whisper implementation, the voice with a different neural model, or the language-model client with a different local runtime, each without changes elsewhere.

Appendix E — Glossary

Term	Meaning
Wake word	A short phrase that signals the assistant to start listening for a command.

Term	Meaning
STT	Speech-to-text; converting spoken audio into written text.
TTS	Text-to-speech; converting written text into spoken audio.
LLM	Large language model; the local model that handles open-ended language.
Rolling window	A fixed-length, continuously updated buffer of the most recent audio.
Risk gate	The mechanism that classifies actions and requires confirmation for risky ones.
SAFE MODE	A mode in which power actions are simulated rather than executed.
Fallback	An alternative backend used automatically when the preferred one is unavailable.
Graceful degradation	Losing quality rather than failing when a component is missing.

Appendix F — Interaction Walkthrough

The following narrative traces a single spoken interaction through every stage of the pipeline, to make the architecture concrete.

The user says, "Hey Jarvis." The microphone callback has been filling the rolling buffer continuously; on the next polling tick the main loop takes a snapshot, notes that its energy is above the floor, and transcribes it. The transcript contains a close match to the wake word, so the detector returns. The assistant speaks a brief acknowledgement and begins capturing the command utterance, recording until the user falls silent.

The user says, "open youtube." The captured audio is transcribed, and any leaked fragment of the wake word is stripped from the front of the transcript. The router walks the command registry in order; the site-opening command matches because the utterance names a known site and carries an open verb rather than a play verb, so it takes precedence over the media command. Its risk level is SAFE, so the risk gate allows it to proceed without confirmation.

The web automation service opens the site in the default browser and returns a short confirmation message. That message is passed to the text-to-speech engine, which synthesises it with the neural voice and plays it through the speaker. The assistant returns to waiting for the wake word, and the whole exchange has touched the microphone, wake detector, speech-to-text engine, router, command registry, risk gate, automation service, and speech engine in turn — the entire architecture exercised by one sentence.

Appendix G — Lessons Learned

Several lessons generalise beyond this project and are worth recording explicitly.

Instrument before tuning

The wake word could not be tuned until the measured input energy and the exact transcript of each window were logged. Once they were, the fix was obvious. Visibility precedes correction.

Fail early, not mid-task

The speech engine's start-up warm-up converts a crash that would have happened in front of the user into a silent fallback at boot. Surfacing latent failures at the earliest safe moment is worth the small extra start-up cost.

Default to inaction for powerful actions

The accidental shutdown taught that a system able to affect the world must treat its most dangerous actions as opt-in. SAFE MODE encodes that lesson structurally.

Isolate the platform's surprises

A blocked media library and thread-bound audio were both platform-specific surprises. Isolating each behind a narrow shim — a stub module, a callback-driven capture — kept the rest of the system oblivious to them.